LeetCode一百题解析

这些都是我个人的理解和经验，毕竟我也是个新手，做这些都是为了更好的理解，如果大家发现有什么问题，欢迎指出改正，谢谢大家

1.两数之和

题目

给定一个整数数组 `nums` 和一个整数目标值 `target`，请你在该数组中找出 **和为目标值** *`target`* 的那 **两个** 整数，并返回它们的数组下标。

你可以假设每种输入只会对应一个答案，并且你不能使用两次相同的元素。

你可以按任意顺序返回答案。

示例 1：

```
输入：nums = [2,7,11,15], target = 9
输出：[0,1]
解释：因为 nums[0] + nums[1] == 9 ，返回 [0, 1] 。
```

示例 2：

```
输入：nums = [3,2,4], target = 6
输出：[1,2]
```

示例 3：

```
输入：nums = [3,3], target = 6
输出：[0,1]
```

提示：

- `2 <= nums.length <= 104`
- `-109 <= nums[i] <= 109`
- `-109 <= target <= 109`
- **只会存在一个有效答案**

进阶：你可以想出一个时间复杂度小于 `O(n2)` 的算法吗？

解析

这道题目的是从给你的一个数组里面找到两个数，这两个数的和要等于目标值。

这样就很清晰我们的思路了

我们可以遍历数组，将遍历的每一个值都与目标值做差，得到的这个值，如果存在于数组中，那么这两个数就满足题目的要求。

下面使用C++实现

```
vector<int> twoSum(vector<int>& nums, int target) {
        int n = nums.size();
        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
                if (nums[i] + nums[j] == target) {
                    return {i, j};
                }
            }
        }
        return {};
    }
```

这个解法是典型的暴力循环解法，能解出来，但是时间复杂度为O(n^2)。

但这个仍然有优化空间，就是使用哈希表

```
vector<int> twoSum(vector<int>& nums, int target) {
        unordered_map<int, int> map;  // key: 数值, value: 下标

        for (int i = 0; i < nums.size(); i++) {
            int need = target - nums[i];  // 需要找的另一个数

            // 如果哈希表里已经有这个数，直接返回结果
            if (map.find(need) != map.end()) {
                return {map[need], i};
            }

            // 没有就把当前数和下标存进哈希表
            map[nums[i]] = i;
        }

        return {};  // 题目保证有解，这里不会执行
    }

```

使用哈希表，可以将时间复杂度从**O(n^2)**降到**O(n)**

这也说明了哈希表是一个对于算法来说很重要的东西，要重点掌握其使用方式，接下来，我们先讲一下这个方法下的代码。

首先，使用**unordered_map<int, int> hash**来生成定义一个哈希表，**<>**里的两个参数分别代表的是**key（键）**的类型和**value(值)**的类型，就如同这里，使用的是**int**类型的**key**和**value**，因为我们要存放的对象是数组的下标和对应的值。

然后我们遍历数组对遍历的每一个值进行计算，算出另外一个符合的值，然后在哈希表里面查询这个值是否存在。如果有就返回，没有就继续找。

这里说一下这个地方

```
if (map.find(need) != map.end()) {
                return {map[need], i};
            }
```

map.find(key)是根据key值来查找的，返回的是一个迭代器，是一个指向**{key, value}** 的一个 “指针”，不仅可以像如上代码使用**map[need]**来取出这个key下对应的value，也可以采用如下办法，

```
auto it = map.find(2);
// 拿到 key
cout << it->first;   // 输出 2
// 拿到 value
cout << it->second;  // 输出 0
```

如果没找到的话，它就会返回**map.end()**,这实际就是一个哈希表结束的标记，如果没找到，**map.find**() 就会返回这个标记，然后和**map.end()**相互比较，如果相等就是没找到，不等就是找到了。

为什么时间复杂度会降低到**O(n)**呢，这是就是因为哈希表的特性了，在哈希表里面找东西，不是用遍历的查找，而是直接锁定到目标位置，而这个直接锁定是怎么实现的呢，是算出来的，这里就不多提及了，感兴趣的可以去单独看看。

这里也简单说一下哈希的使用场景

哈希的核心优势是 **O (1) 时间复杂度的增删查**，是解决 “快速查找、去重、计数” 类问题的首选工具，主要分为以下几类场景：

快速去重与存在性判断

- **典型场景**：判断数组 / 字符串中是否存在重复元素、元素是否出现过、过滤重复数据。
- **代表题目**：最长连续序列、存在重复元素、有效的字母异位词。
- **核心用法**：用 `unordered_set` 存储元素，自动去重，`find` / `count` 操作可 O (1) 判断元素是否存在。

频率统计与计数

- **典型场景**：统计元素出现次数、字符频率、数组中数字的出现次数。
- **代表题目**：两数之和、字母异位词分组、多数元素、字符串中字符出现次数统计。
- **核心用法**：用 `unordered_map` 记录 key-value（元素 - 次数），可快速查询和更新计数。

快速映射与关联查找

- **典型场景**：需要建立元素间的一一映射关系，快速根据一个元素找到对应的另一个元素。
- **代表题目**：两数之和（存差值）、罗马数字转整数、用户 ID 与数据的关联查询。
- **核心用法**：用 `unordered_map` 存储映射关系，O (1) 时间完成查找。

缓存 / 临时存储，避免重复计算

- **典型场景**：动态规划 / 递归中存储已计算的结果（记忆化搜索）、临时存储遍历过的节点 / 状态。
- **代表题目**：带记忆化的斐波那契数列、图的遍历中记录访问过的节点、回溯剪枝。
- **核心用法**：用哈希表存储已计算的结果，避免重复计算，提升效率。

2.字母异位词分组

题目

给你一个字符串数组，请你将 字母异位词 组合在一起。可以按任意顺序返回结果列表。

示例 1:

输入: strs = ["eat", "tea", "tan", "ate", "nat", "bat"]

输出: [["bat"],["nat","tan"],["ate","eat","tea"]]

解释:

- 在 strs 中没有字符串可以通过重新排列来形成 `"bat"`。
- 字符串 `"nat"` 和 `"tan"` 是字母异位词，因为它们可以重新排列以形成彼此。
- 字符串 `"ate"` ， `"eat"` 和 `"tea"` 是字母异位词，因为它们可以重新排列以形成彼此。

示例 2:

输入: strs = [""]

输出: [[""]]

示例 3:

输入: strs = ["a"]

输出: [["a"]]

提示:

- `1 <= strs.length <= 104`
- `0 <= strs[i].length <= 100`
- `strs[i]` 仅包含小写字母

解析

还是一样，我们先来理解字母异位词这个概念。

就是说，这个字符串数组里面的这些字符串，能够通过任意的组合形成彼此的字符串样式，例如“abc”，“cba”，这两个字符串都可以通过变换变成彼此，然后满足这个需求的就可以分为一组。

说人话，就是拥有相同的字母类型和对应个数的字符串就可以分为一组，例如两个字符串里面拥有相同数量的a，相同数量的b，相同数量的c，那么就可以认为这是满足的。

所以我们就有了一个基本思路，对每个字符串进行排序，这样就可以将满足的字符串变成长得一模一样的字符串，然后将满足的字符串下标进行分组，再将这些重新输出为二维数组就可以了。

下面使用C++进行实现：

```
vector<vector<string>> groupAnagrams(vector<string>& strs) {
    //这个地方我们先创建两个动态数组，一个用来存放我们返回的结果，一个存放目标字符串是否已经被分组的布尔值
    vector<vector<string>> res;
    vector<bool> used(strs.size(), false);
```

```
    //这里开始进行遍历
    for (int i = 0; i < strs.size(); i++) {
        if (used[i]) continue;//这里意思是如果已经使用了就直接跳过，不再进行无用的遍历

        //这里创建一个动态的字符串数组来分组，满足的就分到这一组里面
        vector<string> group;
        group.push_back(strs[i]);
        used[i] = true;//标记为已使用

        // 拿到当前字符串排序
        string key_i = strs[i];
        sort(key_i.begin(), key_i.end());

        // 遍历后面所有字符串
        for (int j = i + 1; j < strs.size(); j++) {
            if (used[j]) continue;

            string key_j = strs[j];
            sort(key_j.begin(), key_j.end());

            // 如果排序后一样 → 同一组
            if (key_i == key_j) {
                group.push_back(strs[j]);
                used[j] = true;
            }
        }
        res.push_back(group);
    }
    return res;
}
```

这个仍然采用的是暴力的方式，在我看来，只要你会写暴力的解，那么在其上进行优化也不会太难了。

这个暴力的解法你在LeetCode上提交的话会执行不通过，因为时间超了，时间复杂度为**O ($n^2 \times k \log k$)**，所以我们就可以采取优化，这里我们也可以发现和上一题一样的使用双for来循环对照，所以我们这个题也可以用哈希表来将第二个for给优化了。

下面我们就来使用哈希表来解决

```
vector<vector<string>> groupAnagrams(vector<string>& strs) {

        // 哈希表
        // key: 排序后的字符串
        // value: 同一组的所有原字符串
        unordered_map<string, vector<string>> map;

        // 1. 遍历一次所有字符串 O(n)
        for (string s : strs) {//这里的用法就是遍历strs数组，再把里面的每一个放到s
            // 2. 排序生成 key
            string key = s;
            sort(key.begin(), key.end());

            // 3. 直接放进哈希表对应组里 O(1)
            map[key].push_back(s);
        }
```

```

        // 4. 把哈希表的结果转成二维数组输出
        vector<vector<string>> res;
        for (auto& entry : map) {//这里auto就是自动分类型，map里是啥就是啥类型，然后&就是取引用嘛，会比复制一份出来快一些
            res.push_back(entry.second);
        }

        return res;
    }
```

优化完后，时间复杂度也降低到了**O(n × k log k)**

同时我们也发现了代码也会少一些，这就是算法优化的好处

3.最长连续序列

题目

给定一个未排序的整数数组 `nums` ，找出数字连续的最长序列（不要求序列元素在原数组中连续）的长度。

请你设计并实现时间复杂度为 `O(n)` 的算法解决此问题。

示例 1：

```
输入：nums = [100,4,200,1,3,2]
输出：4
解释：最长数字连续序列是 [1, 2, 3, 4]。它的长度为 4。
```

示例 2：

```
输入：nums = [0,3,7,2,5,8,4,6,0,1]
输出：9
```

示例 3：

```
输入：nums = [1,0,1,2]
输出：3
```

提示：

- `0 <= nums.length <= 105`
- `-109 <= nums[i] <= 109`

解析

先来看题目，在一个没有排序的整数数组里面找出数字的连续最长序列。

数字连续序列是啥呢？

不难理解如0，1，2，3，4，5，6，7，8，9，这样的数字连续就是这个所谓的连续序列。

好了我们知道了要做什么之后，再看题目要求说实现的时间复杂度为**O(n)**,这就说明了它不要你用排序的方法，因为排一次序的时间复杂度为**O(klogk)**,所以不能用

那么，根据前面几道题的思路，我们依然采用哈希的方式来解决，只不过这里使用的是哈希集合。

```
 int longestConsecutive(vector<int>& nums) {
        // 哈希集合：去重 + 快速查找,这里去重说的是哈希集合的自动去重，因为哈希集合不允许重复元素
        //所以在构造哈希集合时会自动去重。
        unordered_set<int> num_set(nums.begin(), nums.end());
        int max_len = 0;

        // 遍历哈希集合中的每个元素
        for (int num : num_set) {
            // 关键：只处理连续序列的起点（num-1 不存在）
            //这里这么做的原因是为了避免重复计算连续序列的长度，如果 num-1 存在，
            //那么说明 num 不是连续序列的起点，已经在之前的迭代中被计算过了。
            //如[1,2,3,4,5,6],当迭代到 num=2 时，num-1=1 存在，所以跳过计算以 2 开头的连续序列
            //这是因为在以1为起点的时候就算过一遍了。
            if (num_set.find(num - 1) == num_set.end()) {
                int current_num = num;
                int current_len = 1;

                // 向后查找连续数字
                while (num_set.find(current_num + 1) != num_set.end()) {
                    current_num++;
                    current_len++;
                }

                // 更新最长长度
                max_len = max(max_len, current_len);
            }
        }
        return max_len;
    }
```

这里有人会有疑问了，有for和while，这是两个循环，怎么时间复杂度仍然是O(n)呢？

我们来举例子说明，例如[1,2,3,5,6,7,8]

在for里，我们以1为起点的时候，访问一次1。

然后在while里面，我们各自访问了一遍后面的连续的数，2，3，这也各是访问一次。

然后就发现这个序列连不上了，又跳回了for。

但是在if的地方截胡了，2，3跳过了。

那么就来到了5，和上面一样的道理，5访问了一次，

然后6，7，8在while里各一次。

发现了什么？

一个集合在这样的for和while下相当于只遍历了一遍。

又有人会问，不是执行了一个if吗，为啥不算执行呢？
不得不说if 判断确实每轮都会执行，但它只是一次极快的条件检查，不会重复遍历、不会重复处理元素；真正消耗时间的是 while 循环的连续查找，而所有元素**只会被 while 访问唯一一次**，不会重复遍历。for 只是全局遍历一遍所有数，后续非起点的元素仅做一次 if 判断就直接跳过，没有额外冗余操作，整体所有元素总共只被完整遍历一轮，所以即便存在 for 嵌套 while，整体时间复杂度依旧是O(n)。

4.移动零

题目

给定一个数组 `nums`，编写一个函数将所有 `0` 移动到数组的末尾，同时保持非零元素的相对顺序。

请注意 ，必须在不复制数组的情况下原地对数组进行操作。

示例 1:

```
输入: nums = [0,1,0,3,12]
输出: [1,3,12,0,0]
```

示例 2:

```
输入: nums = [0]
输出: [0]
```

提示:

- `1 <= nums.length <= 104`
- `-231 <= nums[i] <= 231 - 1`

进阶：你能尽量减少完成的操作次数吗？

解析

题目给了我们一个数组，然后把所有的0都放到数组的末尾后，其余非零的数的原本顺序不能变，还不能用数组赋值的方法。

既然要求0置后，那么我们为何不管0，直接操作其它的数，把这些数按照原有的顺序全部挤到前面去，最后把空余的置为0不就行了吗。

这样，我们可以使用双指针，来对其进行操作。

下面是代码

```
void moveZeroes(vector<int>& nums) {
        int n = nums.size();
```

```
        int i = 0;//这个指针用来记录非零元素应该放置的位置
        for(int j = 0; j < n; j++){
            //j指针用来遍历数组，找到非零元素
            if(nums[j] != 0){
                nums[i] = nums[j];
                i++;
            }
        }
        //将剩余的位置填充为0
        for(int k = i; k < n ;k++){
            nums[k] = 0;
        }
    }
```

除了这个，我们还可以使用交换的方式

```
void moveZeroes(vector<int>& nums) {
        int slow = 0;
        for (int fast = 0; fast < nums.size(); fast++) {
            if (nums[fast] != 0) {
                swap(nums[slow], nums[fast]);
                slow++;
            }
        }
    }
```

这里也可以简单说一下双指针的一些内容。

双指针主要分两类：

1. **同向双指针（快慢 / 滑动窗口）**：两个指针同方向移动，用于原地筛选 / 移动元素（如移动零、去重）、滑动窗口求子串 / 子数组（如最长无重复子串）、链表找环 / 中点。核心是用一个指针筛选，另一个记录有效位置，实现 O (n) 时间、O (1) 空间。
2. **相向双指针（对撞）**：两个指针从两端向中间移动，多用于有序数组的查找（如两数之和）、回文判断。利用有序性，通过和 / 值的大小调整指针，高效缩小范围。

5.盛最多水的容器

题目

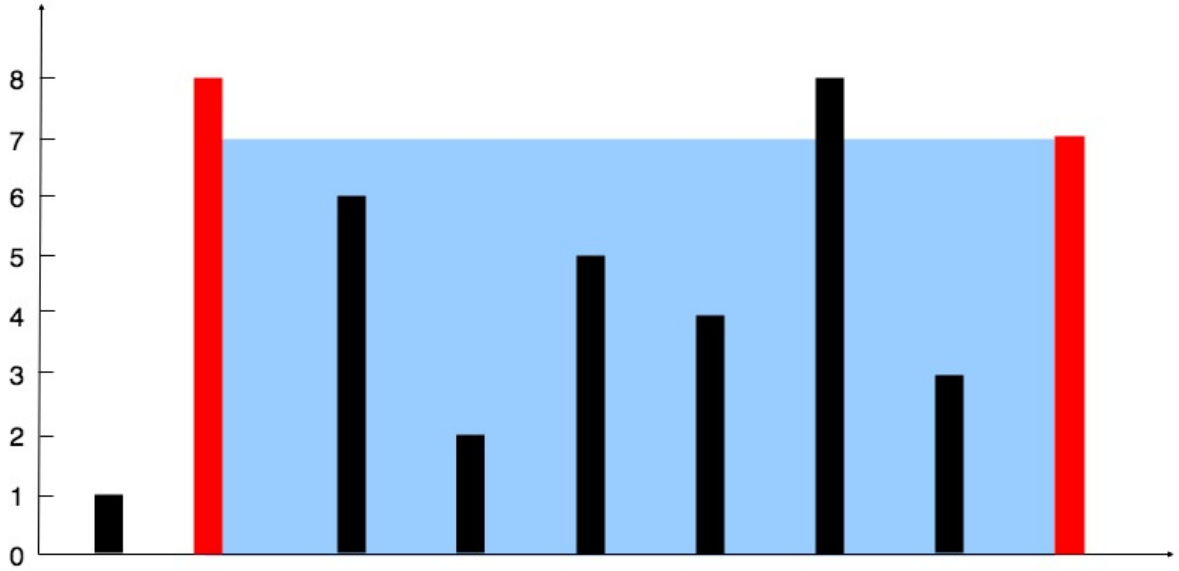
给定一个长度为 `n` 的整数数组 `height` 。有 `n` 条垂线，第 `i` 条线的两个端点是 `(i, 0)` 和 `(i, height[i])` 。

找出其中的两条线，使得它们与 `x` 轴共同构成的容器可以容纳最多的水。

返回容器可以储存的最大水量。

说明：你不能倾斜容器。

示例 1：




```
输入：[1,8,6,2,5,4,8,3,7]
输出：49
解释：图中垂直线代表输入数组 [1,8,6,2,5,4,8,3,7]。在此情况下，容器能够容纳水（表示为蓝色部分）的最大值为 49。
```

示例 2：

```
输入：height = [1,1]
输出：1
```

提示：

- `n == height.length`
- `2 <= n <= 105`
- `0 <= height[i] <= 104`

解析

依旧我们来看题目的要求，说大白话就是找到这个容器的两个边要让里面含的水越多越好，如上题里面提到的双指针，我们就要用到其中的相向双指针，从两边往中间找。

下面是代码实现

```
int maxArea(vector<int>& height) {
    //定义我们的左右指针，分别指向数组的两端
        int left =0;
        int right = height.size()-1;
    //我们要找的最大值
        int maxArea = 0;
    //开始循环，尝试每一种可能的最大情况
        while(left < right){
            //以短边为准算容量
            int area = min(height[left], height[right]) * (right - left);
            //将更大的赋值回去
            maxArea = max(maxArea, area);
            //这里是在调整边界，哪边矮了就调整哪一边，这样才有可能尝试到最大的
```

```
            if(height[left] < height[right]){
                left++;
            }
            else{
                right--;
            }
        }
        return maxArea;
    }
```

6.三数之和

题目

给你一个整数数组 `nums` ，判断是否存在三元组 `[nums[i], nums[j], nums[k]]` 满足 `i != j`、`i != k` 且 `j != k` ，同时还满足 `nums[i] + nums[j] + nums[k] == 0` 。请你返回所有和为 `0` 且不重复的三元组。

注意：答案中不可以包含重复的三元组。

示例 1：

```
输入：nums = [-1,0,1,2,-1,-4]
输出：[[-1,-1,2],[-1,0,1]]
解释：
nums[0] + nums[1] + nums[2] = (-1) + 0 + 1 = 0 。
nums[1] + nums[2] + nums[4] = 0 + 1 + (-1) = 0 。
nums[0] + nums[3] + nums[4] = (-1) + 2 + (-1) = 0 。
不同的三元组是 [-1,0,1] 和 [-1,-1,2] 。
注意，输出的顺序和三元组的顺序并不重要。
```

示例 2：

```
输入：nums = [0,1,1]
输出：[]
解释：唯一可能的三元组和不为 0 。
```

示例 3：

```
输入：nums = [0,0,0]
输出：[[0,0,0]]
解释：唯一可能的三元组和为 0 。
```

提示：

- `3 <= nums.length <= 3000`
- `-105 <= nums[i] <= 105`

解析

三数之和是两数之和的经典升级版，解题本质一脉相承，只是求和对象由两个数字升级为三个数字。

两数之和常规采用**哈希表**快速匹配目标值，时间效率极高；但**三数之和不适合直接使用哈希表**，原因如下：

1. 两数之和只需寻找唯一组合，无需去重；而三数之和要求返回**无重复三元组**，哈希表很难高效处理多层重复元素，去重逻辑极其繁琐，容易产生大量重复结果。
2. 哈希表适合无序数组单层查找，三数之和需要同时约束三个下标互不相同，用哈希表嵌套判断会增加复杂逻辑，代码冗余、易出错。

因此本题放弃纯哈希表思路，改用**排序 + 双指针**最优解法：

先固定第一个元素，把「三数之和为 0」转化为**剩余两数之和等于该数相反数**的两数之和子问题；再借助有序数组的特性，用左右双指针动态收缩查找合法组合，配合相邻重复元素跳过的去重操作，既保证时间复杂度可控，又能天然规避重复三元组。

```cpp
vector<vector<int>> threeSum(vector<int>& nums) {
        vector<vector<int>> res;
        // 先排序，方便去重和双指针
        sort(nums.begin(), nums.end());
        int n = nums.size();

        //这里是三数之和的核心逻辑，外层循环固定一个数，内层使用双指针寻找剩余两个数的和为固定数的相反数
        for (int i = 0; i < n - 2; ++i) {
            // 去重：i 和上一个数相同，跳过,因为上一个数已经处理过了，避免重复结果
            //这里说的重复结果的意思是题目里说的，不管顺序，三元组不能重复，就是不同的三元组里面所含有的数字不能重复
            //和顺序无关，所以如果当前数字和上一个数字相同，就跳过，避免产生重复的三元组结果
            if (i > 0 && nums[i] == nums[i - 1]) continue;

            // 双指针，寻找剩余两个数的和为 -nums[i]
            int j = i + 1;
            int k = n - 1;
            int target = -nums[i];

            while (j < k) {
                int sum = nums[j] + nums[k];
                if (sum == target) {
                    res.push_back({nums[i], nums[j], nums[k]});
                    // j 去重
                    while (j < k && nums[j] == nums[j + 1]) j++;
                    // k 去重
                    while (j < k && nums[k] == nums[k - 1]) k--;
                    j++;
                    k--;
                }
                else if (sum < target) {
                    j++; // 和太小，左指针右移
                }
                else {
                    k--; // 和太大，右指针左移
```

```
                }
            }
        }
        return res;
    }
```

7.接雨水

题目

给定 `n` 个非负整数表示每个宽度为 `1` 的柱子的高度图，计算按此排列的柱子，下雨之后能接多少雨水。

示例 1：

```
输入：height = [0,1,0,2,1,0,1,3,2,1,2,1]
输出：6
解释：上面是由数组 [0,1,0,2,1,0,1,3,2,1,2,1] 表示的高度图，在这种情况下，可以接 6 个单位的雨水（蓝色部分表示雨水）。
```

示例 2：

```
输入：height = [4,2,0,3,2,5]
输出：9
```

提示：

- `n == height.length`
- `1 <= n <= 2 * 104`
- `0 <= height[i] <= 105`

解析

其实这道题很好理解是什么意思，就是说那些又柱子组成的空隙里面能装多少水

下面是第一种思路的解法：

```
int trap(vector<int>& height) {
        int res = 0;
    // 找到最高柱子，决定有多少层
    int max_h = *max_element(height.begin(), height.end());
```

```

    // 逐层计算
    for (int level = 1; level <= max_h; ++level) {
        bool has_left = false;
        int temp = 0;

        for (int h : height) {
            if (h >= level) {
                // 遇到柱子，把前面积累的水加上
                has_left = true;
                res += temp;
                temp = 0;
            } else if (has_left) {
                // 左边有柱子，当前位置可以存水
                temp++;
            }
        }
    }
    return res;
    }
```

这是我第一时间想到的解决方法，就是按层计数。

按层法核心就是把整个柱状图横向一层层切开，从最底层到最高柱子的高度逐层遍历。每一层里从左往右扫描所有柱子，先找到第一个能挡住当前层的左柱子，在这之后出现的空白位置都可以存水，继续往后走，每碰到一根新的柱子，就把这一段中间空白的数量算作当前层接住的雨水，再重新统计下一段空白。一层层全部算完，把每一层的蓄水量累加起来，就是整体一共能接住的雨水总量。

但这个就是有很明显的局限性了。

按层法的时间复杂度为 ***O(n×maxH)***，**n**是柱子数量，**maxH** 为最大柱子高度，需要依据最大高度逐层遍历整个数组，存在大量重复扫描；该方法最大局限是极易受柱子最大高度影响，当柱子高度数值极大时，运算次数会急剧增加、运行效率大幅下降，容易超时，无法处理大规模数据，不能满足算法题的性能要求。

所以我们还是要用双指针来优化。

```
int trap(vector<int>& height) {
    int left = 0;
    int right = height.size() - 1;
    int leftMax = 0;
    int rightMax = 0;
    int res = 0;

    while (left < right)
    {
        //左边的墙小于右边的墙才有可能有水的积蓄
        if (height[left] < height[right])
        {
            //如果左指针指向的这个墙比左边的最高墙还高，那么就更新左边的最高墙
            if (height[left] >= leftMax)
                leftMax = height[left];
            //同样的道理，如果矮了，就说明这地方可以积水
            else
                res += leftMax - height[left];
            left++;
```

```
        }
        //右边的道理和左边的一样
        else
        {
            if (height[right] >= rightMax)
                rightMax = height[right];
            else
                res += rightMax - height[right];
            right--;
        }
    }
    return res;
}
```

总而言之，双指针法从数组**左右两端**向中间遍历，分别维护左侧、右侧已遍历区域的最高柱子高度。每次比较左右指针指向的柱子高度，**较矮一侧的柱子所在位置可直接计算接水量**（该位置的积水由同侧最高柱子决定，另一侧必然有更高柱子挡水）；若当前柱子高于同侧最高高度，则更新最大高度，计算完后将较矮一侧的指针向中间移动，直到两指针相遇，累加所有位置的接水量即为结果。

并且两个指针全程只向中间移动，**最多遍历数组一次**，没有嵌套循环，执行次数与数组长度成正比，时间复杂度减小到了**O(n)**

8.无重复字符的最长子串

题目

给定一个字符串 `s` ，请你找出其中不含有重复字符的 **最长 子串** 的长度。

示例 1:

```
输入: s = "abcabcbb"
输出: 3
解释: 因为无重复字符的最长子串是 "abc"，所以其长度为 3。注意 "bca" 和 "cab" 也是正确答案。
```

示例 2:

```
输入: s = "bbbbb"
输出: 1
解释: 因为无重复字符的最长子串是 "b"，所以其长度为 1。
```

示例 3:

```
输入: s = "pwwkew"
输出: 3
解释: 因为无重复字符的最长子串是 "wke"，所以其长度为 3。
      请注意，你的答案必须是 子串 的长度，"pwke" 是一个子序列，不是子串。
```

提示:

- `0 <= s.length <= 5 * 104`

- `s` 由英文字母、数字、符号和空格组成

解析

```
 int lengthOfLongestSubstring(string s) {
        // 哈希表记录每个字符上一次出现的位置
        /*这里为什么要这么使用，是因为vector<int> last(256, -1) 是 C++ 标准库的动态数组，
        它自动初始化所有元素为 -1，代码更简洁、安全。
        题目说字符串 s 由 英文字母、数字、符号和空格 组成，这些都属于 ASCII 字符（范围 0~127）。
        为了安全（也能覆盖扩展 ASCII），我们直接用 0~255 共 256 个可能的字符值。
        这样我们可以直接用字符的 ASCII 码作为下标，快速存取该字符上一次出现的位置。
        */
        vector<int> last(256, -1);

        int left = 0;               // 滑动窗口左边界
        int maxLen = 0;             // 最长无重复子串长度

        for (int right = 0; right < s.size(); ++right) {
            char c = s[right];
            // 如果当前字符已经在窗口中出现过，则更新左边界
            if (last[c] >= left) {
                left = last[c] + 1;
            }
            // 记录当前字符的位置
            last[c] = right;
            // 更新最长长度
            maxLen = max(maxLen, right - left + 1);
        }
        return maxLen;
    }

```

这个算法用滑动窗口维护无重复子串，并用一个数组记录每个字符的最新位置，当遇到重复字符时，直接将窗口左边界跳到该字符上一次出现位置的下一个位置，从而在 O(n) 时间内找到最长无重复子串的长度。

这里简单提一下滑动窗口相关的。

滑动窗口其实也是一种**双指针**，只是这种技巧专门解决**连续子串/子数组**的优化问题。

例如

1. **最长无重复子串**（该题）
2. **长度最小子数组**（和 ≥ target）
3. **固定大小窗口的最大/最小值**
4. **字符串排列 / 字母异位词**
5. **最多替换 k 个字符后最长连续串**